0853. Car Fleet

culty: Medium | Category: Stack | LeetCode: https://leetcode.com/problems/car-f

Problem

- Cars at position[i] with speed[i] drive toward target. A faster car that catches a slower one ahead joins it as a fleet (moving at the slower speed). How many fleets arrive?
- Example: target 12, position [10,8,0,5,3], speed [2,4,1,1,3] → 3.

Intuition

- Compute each car's arrival time if it were alone: (target - pos) / speed.
- Process cars from closest to target backwards. A car behind with time <= the fleet ahead catches it and merges; a larger time starts a new fleet.

Brute Force

- Simulate time steps: slow, and floating-point merges get messy.

Optimal Approach

- Sort cars by position descending.
- Keep the time of the last fleet; for each car t = (target-p)/s.
- If t > lastFleetTime, it is a new fleet: count++, lastFleetTime = t.

Worked Example

```
target 12.

pos 10 speed 2 -> t=1    new fleet (1)
pos  8 speed 4 -> t=1    <=1 merges
pos  5 speed 1 -> t=7    new fleet (2)
pos  3 speed 3 -> t=3    <=7 merges
pos  0 speed 1 -> t=12   new fleet (3)
```

Java Solution

```java
public class CarFleet {
    public int carFleet(int target, int[] position, int[] speed) {
        int n = position.length;
        Integer[] idx = new Integer[n];
        for (int i = 0; i < n; i++) idx[i] = i;
        Arrays.sort(idx, (a, b) -> position[b] - position[a]);
        int fleets = 0;
        double last = -1;
        for (int i : idx) {
            double t = (double) (target - position[i]) / speed[i];
            if (t > last) { fleets++; last = t; }
        }
        return fleets;
    }
}
```

Complexity

- Time: O(n log n) for sorting. Space: O(n).

Pitfalls

- Use double for the time; integer division gives wrong merges.
- Catching exactly at the target still counts as one fleet (<=).